INFORMATION RETRIEVAL

- SISE 4406, Fall 2023 -

Text operations & Indexing

Chapters 2 & 4

Some slides adapted from: Christopher Manning, Pandu Nayak and Prabhakar Raghavan

© Nir Grinberg

Outline

Text operations

- Normalization
  - Stemming methods

Indexing

1. Simple index construction
2. Massive data indexing
3. Distributed & dynamic indexing

© Nir Grinberg

Stemming methods

- Table Lookup
- N-gram
- Affix removal (suffix, prefix)
  - Simple removal – Porter
  - Longest match – Lovins (not covered)

© Nir Grinberg

Stemming Methods (#1): Table lookup

- Use a predefined table for stemming

TERM	STEM
Teaching	teach
Teaches	teach
Taught	teach
Teacher	teach

© Nir Grinberg

Stemmer - table lookup

- Disadvantages
  - A stemmer table for English does not exist.
  - Storage overhead.
- Advantage
  - Easy to implement, efficient (search time)
  - Could work well for static collections.

© Nir Grinberg

Stemming methods (#2): N-Gram stemmers

- Terms are broken to n consecutive characters (n=2, pairs of letters)
- Association measures are calculated between pairs of terms, based on shared unique diagrams.
- Algorithm: For each new processed term, check if it is a stem of an existing term, otherwise add to index.

Stemmers - N-Gram stemmers

- Example
  - statistics : st ta at ti is st ti ic cs
  - unique digrams: **ta at** cs **ic is st ti**

  - statistical : st ta at ti is st ti ic ca al
  - unique digrams: al **at** ca **ic is st ta ti**

  - 6 shared digrams

Similarity Measures - Dice

Dice's coefficient (similarity)

$$S = \frac{2C}{A+B} = \frac{2*6}{7+8} = .80$$

A and B are the numbers of unique digrams in the first and the second words. C is the number of unique digrams shared by A and B.

© Nir Grinberg

Similarity option 2- Jaccard coefficient

- A commonly-used measure of overlap
- Let X and Y be two sets; then the J.C. is

$$\left|X \cap Y\right| / \left|X \cup Y\right|$$

- Equals 1 when X and Y have the same elements and zero when they are disjoint
- X and Y don't have to be of the same size
- Our example : 6/9 = 0.66 (9 is the number of the overall unique grams)
- Always assigns a number between 0 and 1
  - Now threshold to decide if you have a match
  - E.g., if J.C. > 0.8, declare a match

© Nir Grinberg

Stemming methods (#3): Affix Removal

- Algorithms to remove suffixes and/or prefixes form letters leaving a stem
- Example of rules:
  - if a word ends in “ies” but not eies or “aies”
    - then “ies” -> “y” (studies-> study)
  - if a word ends in “es” but not “aes”, “ees” or “oes”
    - then “es”-> “e” (tables-> table, referees not->refere)

Stemmers Affix Removal

- Longest match vs. simple rules
  - In longest match removes the longest possible string of characters
- Porter's Algorithm - uses a suffix list for suffix stripping.
- http://www.tartarus.org/~martin/PorterStemmer/

Porter's algorithm

- Commonest algorithm for stemming English
  - Results suggest it's at least as good as other stemming options
- Conventions + 5 phases of reductions
  - phases applied sequentially
  - each phase consists of a set of commands
  - sample convention: *Of the rules in a compound command, select the one that applies to the longest suffix.*

Typical rules in Porter

- *sses* → *ss*
- *ies* → *i*
- *ational* → *ate*
- *tional* → *tion*

- Weight of word sensitive rules
- *(m>1) EMENT* →
  - *replacement* → *replac*
  - *cement* → *cement*

Porter's stemming algorithm

7.

- Based on a measure of vowel-consonant sequences
  - measure **m** for a stem is **[C](VC)^m[V]** where **C** is a sequence of consonants and **V** is a sequence of vowels (including "y") (**[ ]** indicates optional)
  - **m=0** (tree, by), **m=1** (trouble, oats, trees, ivy), **m=2** (troubles, private)
- Some Notation:
  - *<X> --> stem ends with letter X
  - *v* --> stem contains a vowel
  - *d --> stem ends in double consonant
  - *o --> stem ends with a **cvc** sequence where the final consonant is not w, x, y
- Algorithm is based on a set of condition action rules
  - old suffix --> new suffix
  - rules are divided into steps and are examined in sequence
- Good average recall and precision

- A selection of rules from Porter's algorithm:

STEP	CONDITION	SUFFIX	REPLACEMENT	EXAMPLE
1a	NULL	sses	ss	stresses -> stress
	NULL	ies	I	ponies -> poni
	NULL	ss	ss	caress -> caress
	NULL	s	NULL	cats -> cat
1b	*v*	ing	NULL	making -> mak
	. . .	. . .	. . .	. . .
1b1	NULL	at	ate	inflat(ed) -> inflate
	. . .	. . .	. . .	. . .
1c	*v*	y	I	happy -> happi
2	m > 0	aliti	al	formaliti > formal
	m > 0	izer	ize	digitizer -> digitize
	. . .	. . .	. . .	. . .
3	m > 0	icate	ic	duplicate -> duplic
	. . .	. . .	. . .	. . .
4	m > 1	able	NULL	adjustable -> adjust
	m > 1	icate	NULL	microscopic -> microscop
	. . .	. . .	. . .	. . .
5a	m > 1	e	NULL	inflate -> inflat
	. . .	. . .	. . .	. . .
5b	M > 1, *d, *<L>	NULL	single letter	controll -> control, roll -> roll

Other stemmers

- Other stemmers exist:
  - Lovins stemmer
    - http://www.comp.lancs.ac.uk/computing/research/stemming/general/lovins.htm
    - Single-pass, longest suffix removal (about 250 rules)
  - Paice/Husk stemmer
  - Snowball
- Full morphological analysis (lemmatization)
  - At most modest benefits for retrieval

Stemming methods: Problems

- Overstemming - e.g. redim -> red
- Understemming: e.g. users -> user
- Errors of "comission":
  - organization, organ → organ
  - Executive, execute -> execut
  - arm, army → arm
  - University, universe -> univers
- Errors of "omission":
  - cylinder, cylindrical
  - create, creation
  - Europe, European

- Accuracy- e.g. skies -> sky not ski. A special rule for k in plurals is needed.

- How can a stemmer be evaluated? What measures should be used?

ANY QUESTIONS ABOUT STEMMING ?

© Nir Grinberg

Outline

~~Text operations~~

- ~~Normalization~~
  - ~~Stemming methods~~

Indexing

1. Simple index construction
2. Massive data indexing
3. Distributed & dynamic indexing

1. Simple index construction

- Inverted index
  - Construction & retrieval
  - Useful data structures
  - Compression
- Indexing decisions:
  - Vocabulary
  - Heaps' law
  - Phrase queries

Indexes

- *Indexes* are data structures designed to make search faster
- Text search has unique requirements, which leads to unique data structures
- Most common data structure is *inverted index*
  - general name for a class of structures
  - "inverted" because documents are associated with words, rather than words with documents

Inverted Index

Index representation

1: Winter is coming.
2: Ours is the fury.
3: The choice is yours.

term	freq	documents
choice	1	3
coming	1	1
fury	1	2
is	3	1, 2, 3
ours	1	2
the	2	2, 3
winter	1	1
yours	1	3

Dictionary

Postings

Inverted index

Simple in-memory construction

For each document:

- Tokenize the document
- Record which tokens occurred in this document
  - Update dictionary (if needed) with new pointer to postings file
  - Create/update postings file. May save extra info (e.g., position, font, type, tf-idf, etc..)

© Nir Grinberg

Inverted index

Retrieval algorithm

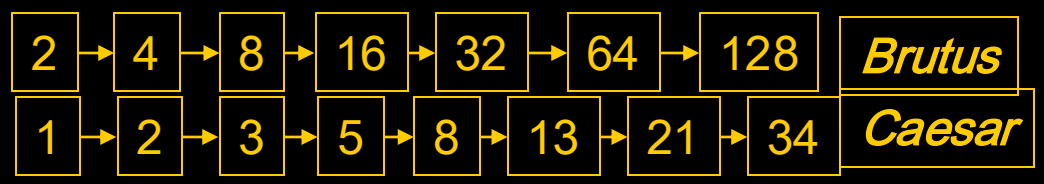
1. Vocabulary search.
2. Determination of the answer set.
3. Apply the retrieval function on the matching documents.

© Nir Grinberg

Processing AND queries

Consider processing the query: ***Brutus** AND **Caesar***

1. Locate ***Brutus*** in the Dictionary and retrieve its postings.
2. Locate ***Caesar*** in the Dictionary and retrieve its postings.
3. Merge the two postings (intersect the document sets):

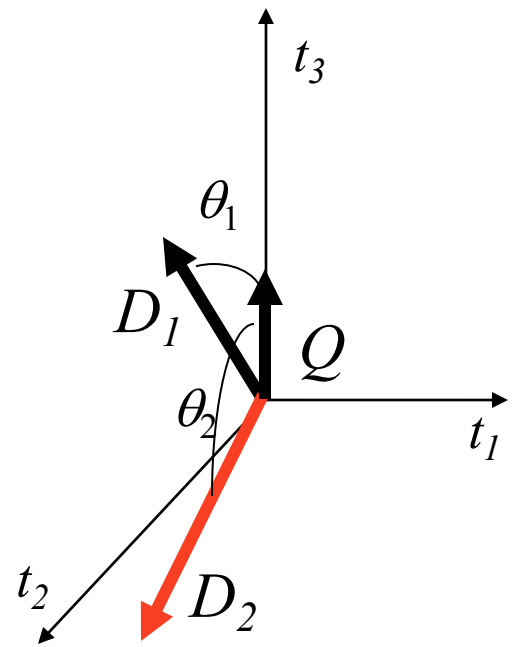


Crucial that posting are **sorted**.

Ranking using Cosine Similarity

- Cosine similarity measures the cosine of the angle between the query vector and each document vector
-

$$\mathrm{CosSim}(\boldsymbol{d}_j, \boldsymbol{q}) = \frac{\vec{d}_j \cdot \vec{q}}{\left|\vec{d}_j\right| \cdot \left|\vec{q}\right|} = \frac{\sum_{i=1}^{t} (w_{ij} \cdot w_{iq})}{\sqrt{\sum_{i=1}^{t} w_{ij}^{2} \cdot \sum_{i=1}^{t} w_{iq}^{2}}}$$



© Nir Grinberg

Naïve retrieval

Algorithm:

```
initialize all sim(q, d_i) = 0;
for each document di (i = 1, …, n)
    { for each term t_j (j = 1, …, m)
        if t_j appears in both q and d_i
            sim(q, d_i) += q_j *w_ij;
      sim(q, d_i) = sim(q, d_i)*NF[i] *(1/|q|); }
sort documents in descending similarities and
    display the top k to the user;
```


© Nir Grinberg

Inverted Files

Observation: Method 1 is not efficient as most non-zero entries in the document-term matrix need to be accessed.

Method 2: Use Inverted File Index

Notation:

$I(t_j)$ – posting list for term t_j

NF(i) – Normalization factor for doc i == $1/|d_i|$

$sim(q, d_i)$ – vector/list of similarity scores

© Nir Grinberg

Retrieval using Inverted index

Method 2: Use Inverted Index

```
initialize all sim(q, d_i) = 0;
for each term t_j in q
   { find I(t) using the hash table;
     for each (d_i, w_ij) in I(t)
          sim(q, d_i) += q_j *w_ij; }
for each document di
     sim(q, d_i) = sim(q, d_i)*(1/|q|)*nf[i];
sort documents in descending similarities
        and display the top k to user
```


© Nir Grinberg

Efficient Retrieval: Example

q = { (t1, 1), (t3, 1) }, 1/|q| = 0.7071

d1 = { (t1, 2), (t2, 1), (t3, 1) }, nf[1] = 0.4082

d2 = { (t2, 2), (t3, 1), (t4, 1) }, nf[2] = 0.4082

d3 = { (t1, 1), (t3, 1), (t4, 1) }, nf[3] = 0.5774

d4 = { (t1, 2), (t2, 1), (t3, 2), (t4, 2) }, nf[4] = 0.2774

d5 = { (t2, 2), (t4, 1), (t5, 2) }, nf[5] = 0.3333

I(t1) = { (d1, 2), (d3, 1), (d4, 2) }

I(t2) = { (d1, 1), (d2, 2), (d4, 1), (d5, 2) }

I(t3) = { (d1, 1), (d2, 1), (d3, 1), (d4, 2) }

I(t4) = { (d2, 1), (d3, 1), (d4, 1), (d5, 1) }

I(t5) = { (d5, 2) }

© Nir Grinberg

Efficient Retrieval: Example

After t1 is processed:

sim(q, d1) = 2, sim(q, d2) = 0, sim(q, d3) = 1

sim(q, d4) = 2, sim(q, d5) = 0

After t3 is processed:

sim(q, d1) = 3, sim(q, d2) = 1, sim(q, d3) = 2

sim(q, d4) = 4, sim(q, d5) = 0

After normalization:

sim(q, d1) = .87, sim(q, d2) = .29, sim(q, d3) = .82

sim(q, d4) = .78, sim(q, d5) = 0

© Nir Grinberg

Retrieval using inverted indices

Some observations about Method 2:

- **If a document d does not contain any term of a given query q, then d will not be involved in the evaluation of q.**
- **Only non-zero entries in the columns in the document-term matrix corresponding to the query terms are used to evaluate the query.**
- **computes the similarities of multiple documents simultaneously (w.r.t. each query word)**

© Nir Grinberg

Useful data structures

- Dictionary/Global statistics:
  - Hashtable
  - Trees (binary, B-tree)
- Posting lists:
  - Sorted arrays. Why not lists?

Data Structures for Posting lists

- Sorted Arrays –
  - Cheap in space
  - Competitive in performance O(logn) when binary search is used.
  - Expensive in updating.
  - Easy to implement.
  - Very broadly used.

Dictionary data structures

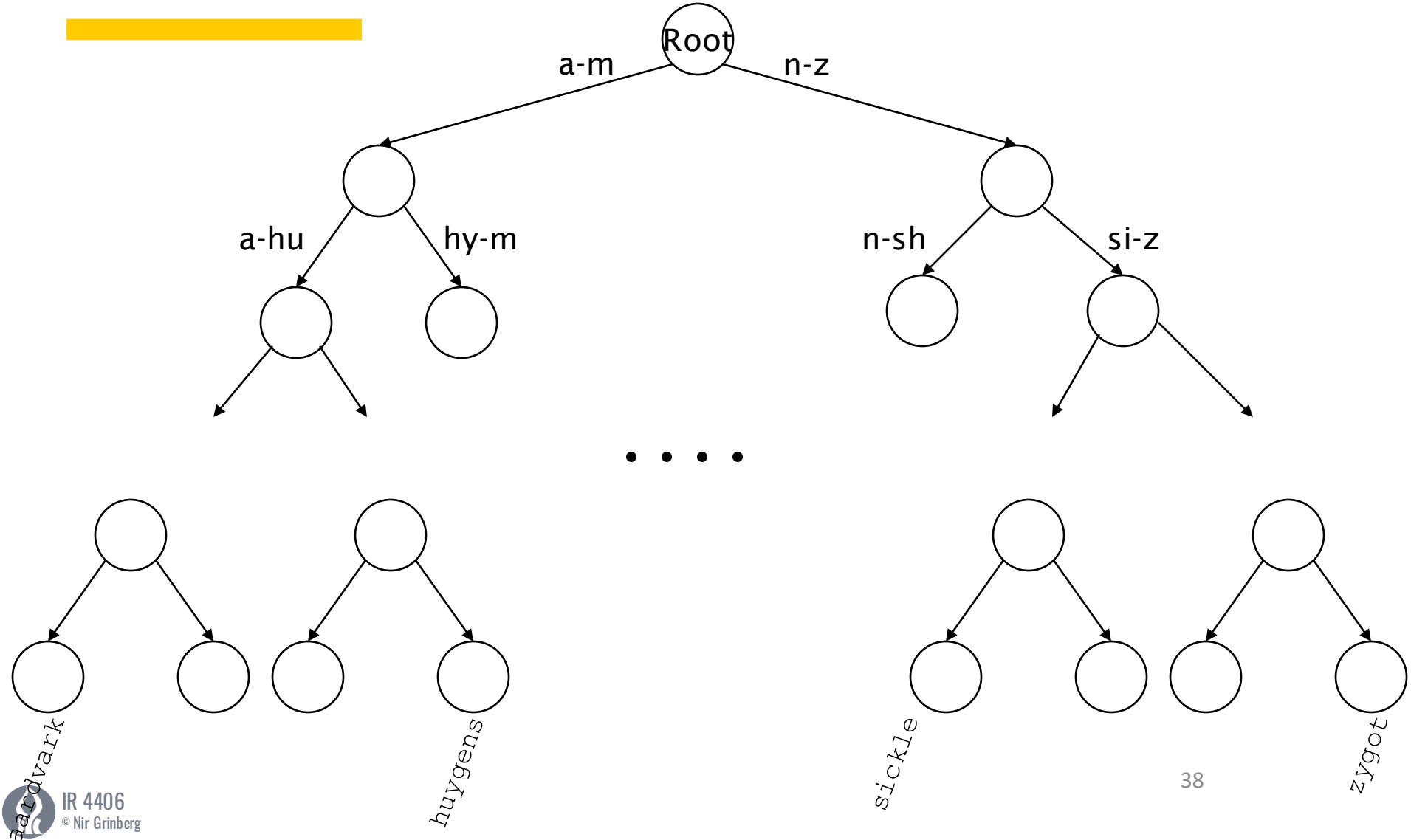
- Two other choices:
  - Hashtables
  - Trees
- Some IR systems use hashtables, some trees

Hashtables

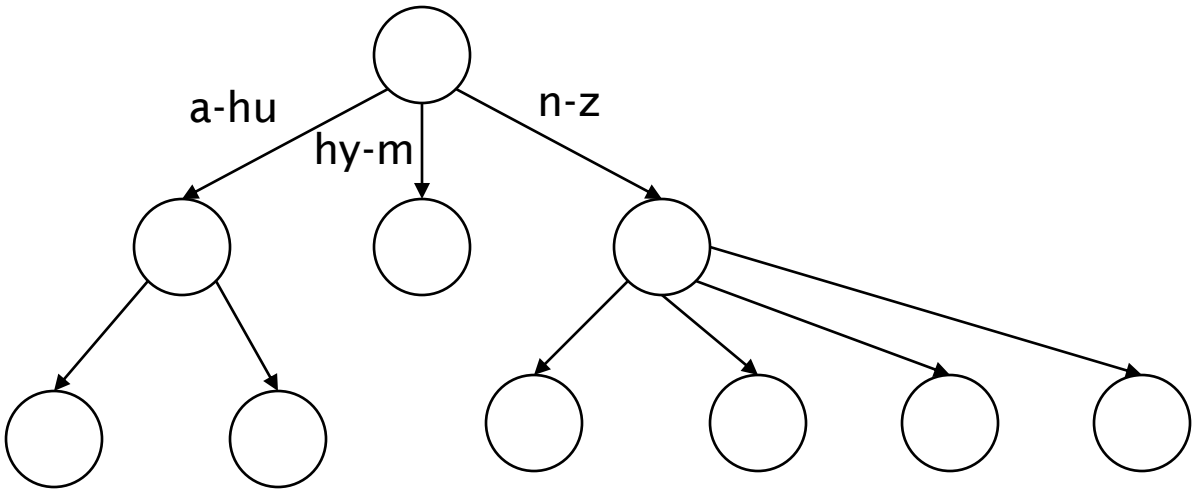
- Each vocabulary term is hashed to an integer
  - (We assume you've seen hashtables before)
- Pros:
  - Lookup is faster than for a tree: O(1)
- Cons:
  - No easy way to find minor variants:
    - judgment/judgement
  - No prefix search [tolerant retrieval]
  - If vocabulary keeps growing, need to occasionally do the expensive operation of rehashing *everything*

© Nir Grinberg

Tree: binary tree



Tree: B-tree



- Definition: Every internal nodel has a number of children in the interval [*a*,*b*] where *a, b* are appropriate natural numbers, e.g., [2,4].

Trees

- Simplest: binary tree
- More usual: B-trees
- Trees require a standard ordering of characters and hence strings … but we typically have one
- Pros:
  - Solves the prefix problem (terms starting with *hyp*)
- Cons:
  - Slower: O(log *M*) [and this requires *balanced* tree]
  - Rebalancing binary trees is expensive
    - But B-trees mitigate the rebalancing problem

Index Compression

- The dictionary → RAM
- The posting lists → Disk
- Posting files ~30%-40% of the text size with decent compression.
- Useful compression technique:
  - interval difference
  - Common numbers are encoded using fewer bits
  - Does not significantly degrade performance.
- Chapter 5 in the book, self-studied

1. Simple index construction

- ~~Inverted index~~
  - ~~Construction & retrieval~~
  - ~~Useful data structures~~
  - ~~Compression~~
- Indexing decisions:
  - Vocabulary
  - Heaps' law
  - Phrase queries

Indexing decisions

- Types of terms to include
  - Restriction on stoplist, spaces, punctuation, digits, standard prefixes etc.
  - Single words or phrases
  - Synonyms, semantic related words, morphological variations, known vocabulary
- What kind of information to hold
  - Pointer to the document
  - Location of word (in the posting file)
  - Frequency in document, in collection (index or posting)
  - In search engines- information held about an occurrence of a term and on a page is very detailed

Indexing decisions

- Trade-off between coverage and specificity
- Vocabulary is key!
  - How does it grow with more documents?
  - Not upper-bounded
  - Can use stemming + pruning of insignificant words to reduce size

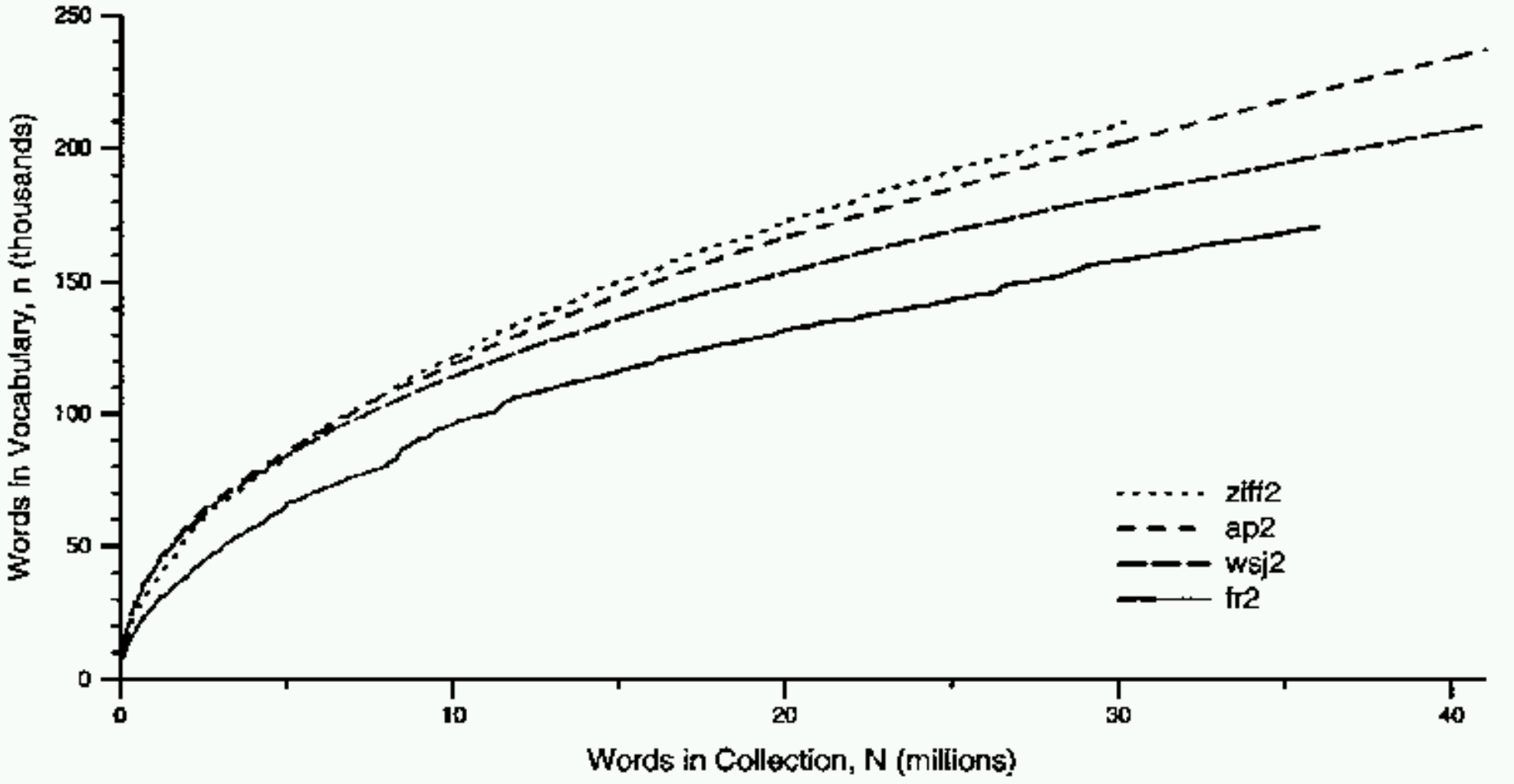
Heaps' Law

- If V is the size of the vocabulary and the n is the length of the corpus in words:

$$V = Kn^{\beta} \quad \text{with constants } K,\ 0 < \beta < 1$$

- Typical constants:
  - $K \approx 10–100$
  - $\beta \approx 0.4–0.6$ (approx. square-root)

Heaps' Law Data



Phrase queries

- E.g. “Ben-Gurion University”
- Why is this important?
- How do we support this?

© Nir Grinberg

A first attempt: Biword indexes

- Index every consecutive pair of terms in the text as a phrase
- For example the text "Friends, Romans, Countrymen" would generate the biwords
  - ***friends romans***
  - ***romans countrymen***
- Each of these biwords is now a dictionary term
- Two-word phrase query-processing is now immediate.

© Nir Grinberg

Issues for biword indexes

- Index blowup due to bigger dictionary
  - Infeasible for more than biwords, big even for them
- But can be part of a compound strategy

Phrase queries – solution #2

- E.g. "Ben-Gurion University"
- Why is this important?
- How do we support this?
- **Positional index**:

  <*term*, # docs containing *term*;

  doc1: [position1, position2 ...] ;

  doc2: [position1, position2 ...] ;

  etc.>

Impacts of positional index

- A positional index expands postings storage ***substantially***
  - Roughly 2-4 times in size for English
- Runtime cost. Why?
- But:
  - Good compression mitigate this somewhat
  - Useful for phrase queries AND proximity queries, which are really important
  - → Positional indices are now standard

© Nir Grinberg

ANY QUESTIONS ABOUT BASIC INDEX STRUCT?

© Nir Grinberg

Outline

~~Text operations~~

- ~~Normalization~~
  - ~~Stemming methods~~

Indexing

1. ~~Simple index construction~~
2. Massive data indexing
3. Distributed & dynamic indexing

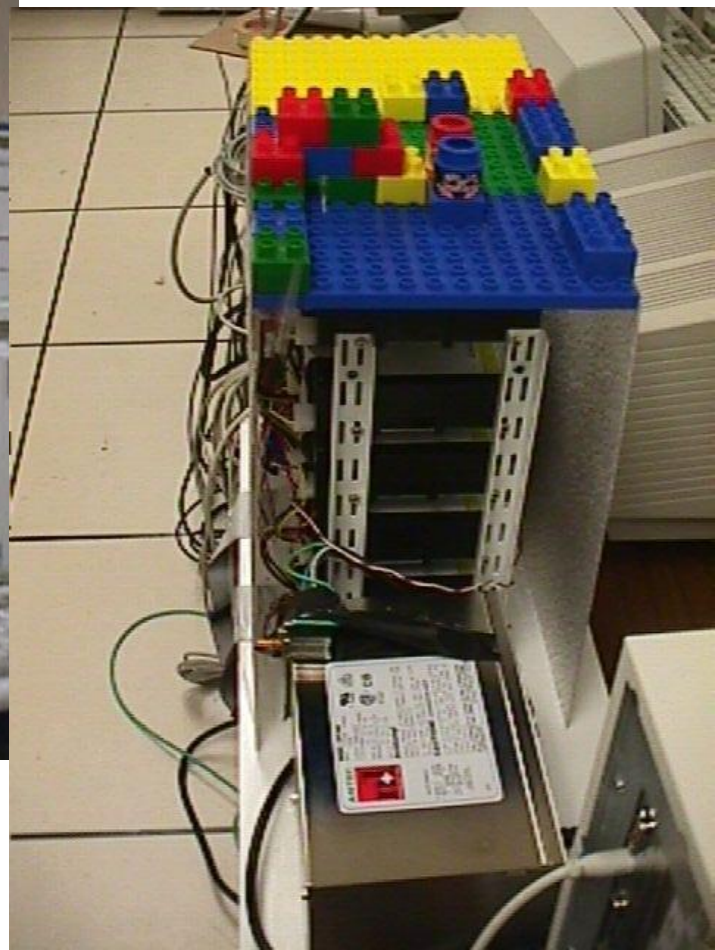
Hardware basics

- Access to data in memory is ***much*** faster than access to data on disk.
- Disk seeks: No data is transferred from disk while the disk head is being positioned.
- Therefore: Transferring one large chunk of data from disk to memory is faster than transferring many small chunks.
- Disk I/O is block-based: Reading and writing of entire blocks (as opposed to smaller chunks).
- Block sizes: 8KB to 256 KB.

© Nir Grinberg

Google' Hardware 1998

Homemade disk box with ten 9GB SCSI drives



Sun Ultra II with dual 200MHz processors, and 256MB of RAM

© Nir Grinberg

Hardware assumptions for this lecture

- Average seek time
  - 5×10^{-3} s = 5 ms
- Transfer time per byte
  - 2×10^{-8} s = 0.02 μs
- Processor's clock rate
  - 10^9 operation per second
  - low-level operation (e.g., compare & swap a word): 10^{-8} s = 0.01 μs
- size of main memory
  - a few GB
- size of disk space
  - ~1 TB or more

© Nir Grinberg

A Reuters RCV1 document

REUTERS

You are here: Home > News > Science > **Article**

Go to a Section: U.S. International Business Markets Politics Entertainment Technology Sports Oddly Enoug

Extreme conditions create rare Antarctic clouds

Tue Aug 1, 2006 3:20am ET

Email This Article | Print This Article | Reprints

[-] Text [+]

SYDNEY (Reuters) - Rare, mother-of-pearl colored clouds caused by extreme weather conditions above Antarctica are a possible indication of global warming, Australian scientists said on Tuesday.

Known as nacreous clouds, the spectacular formations showing delicate wisps of colors were photographed in the sky over an Australian meteorological base at Mawson Station on July 25.

Reuters RCV1 statistics

symbol	statistic	value
N	documents	800,000
L	avg. # tokens per doc	200
M	terms (= word types)	400,000
	avg. # bytes per token (incl. spaces/punct.)	6
	avg. # bytes per token (without spaces/punct.)	4.5
	avg. # bytes per term	7.5
	non-positional postings	**100,000,000**

(Taking hardware performance into consideration)

HOW DO YOU SORT 100M POSTING ENTRIES?

Bottleneck

- Parse and build postings entries one doc at a time
- Now sort postings entries by term (then by doc within each term)
- Doing this with random disk seeks would be too slow – must sort T=100M records

If every comparison took 2 disk seeks, and N items could be sorted with $N \log_2 N$ comparisons, how long would this take?

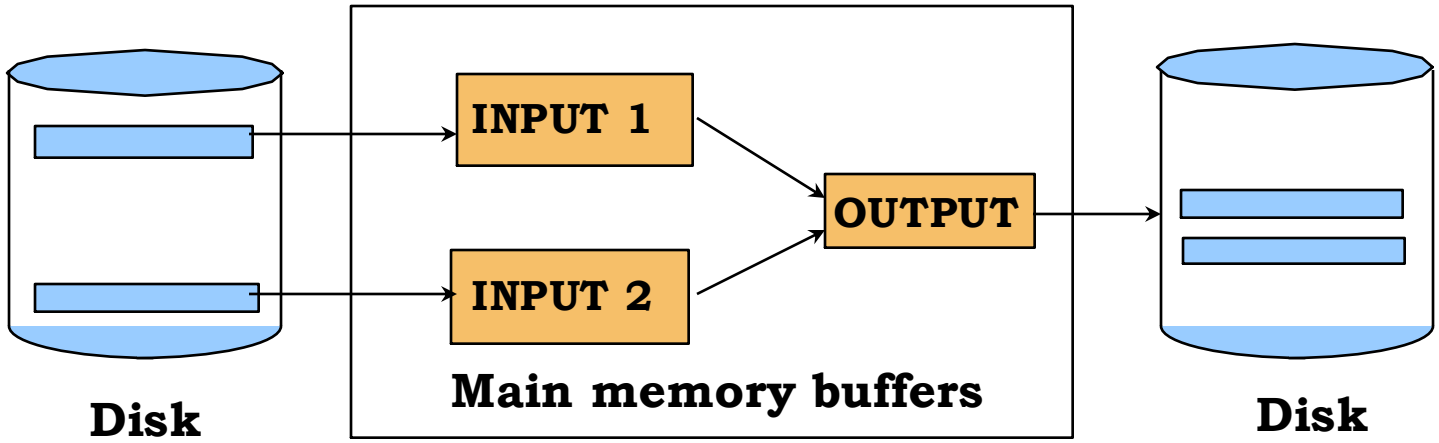
External Sorting

- Problem: Sort 100Gb of data with 1Gb of RAM.
- When a file doesn't fit in memory, there are two stages in sorting:
  1. File is divided into several segments, each of which sorted separately
  2. Sorted segments are merged

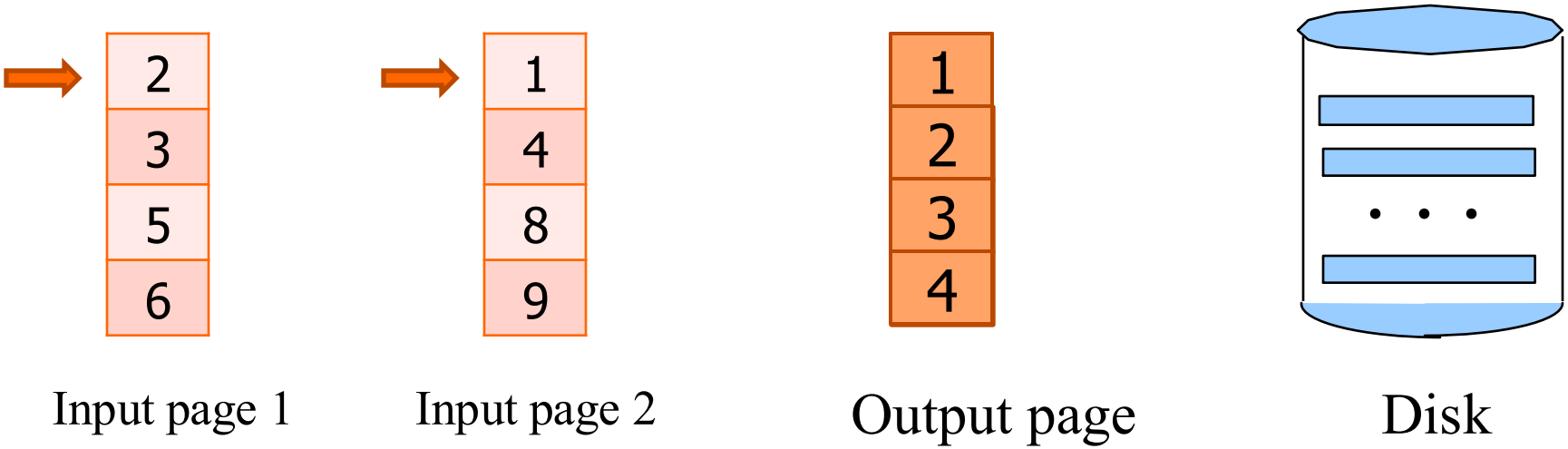
Each stage involves reading and writing the file at least once!

2-Way Sort: Requires 3 Buffers

- Pass 1: Read a page, sort it, write it.
  - only one buffer page is used
- Pass 2, 3, …, etc.:
  - three buffer pages used.



Main Memory 2-Way Merge Sort



- Settings:
  - A frame holds 4 keys.

Two-Way External Merge Sort

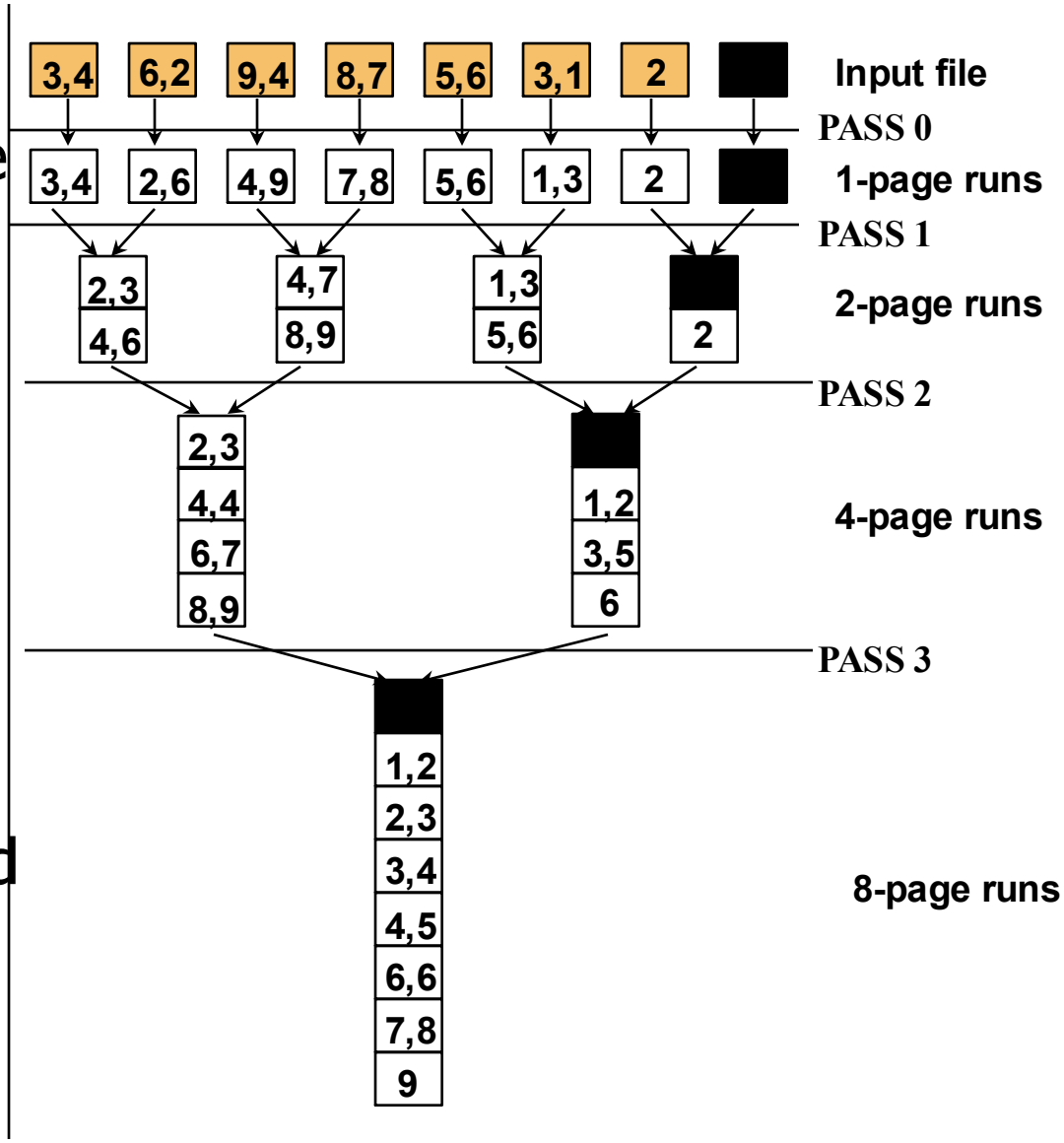
- Each pass we read + write each page in file.
- N pages in the file => the number of passes

$$= \lceil \log_2 N \rceil + 1$$

- So total cost is:

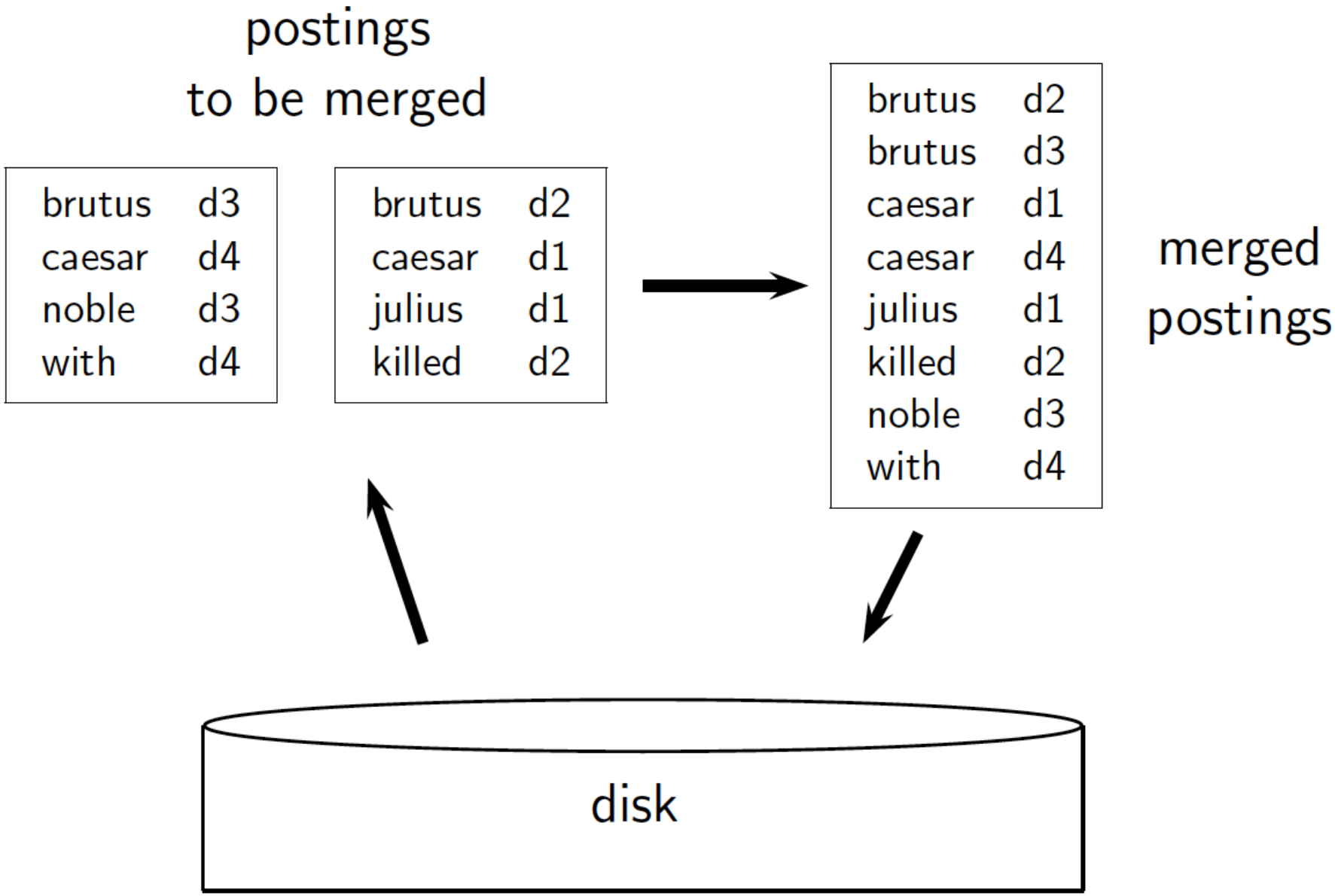
$$2N\left(\lceil \log_2 N \rceil + 1\right)$$

- *Idea:* ***Divide and conquer:*** sort subfiles and merge



BSBI: Blocked sort-based Indexing (Sorting with fewer disk seeks)

- 12-byte (4+4+4) records *(termid, docid, freq).*
- These are generated as we parse docs.
- Must now sort 100M such 12-byte records by *term*.
- Define a Block ~ 10M such records
  - Can easily fit a couple into memory.
  - Will have 10 such blocks to start with.
- Basic idea of algorithm:
  - Accumulate postings for each block, sort, write to disk.
  - Then merge the blocks into one long sorted order.


postings
to be merged
brutus d3
caesar d4
noble d3
with d4
brutus d2
caesar d1
julius d1
killed d2
brutus d2
brutus d3
caesar d1
caesar d4
julius d1
killed d2
noble d3
with d4
merged
postings
disk

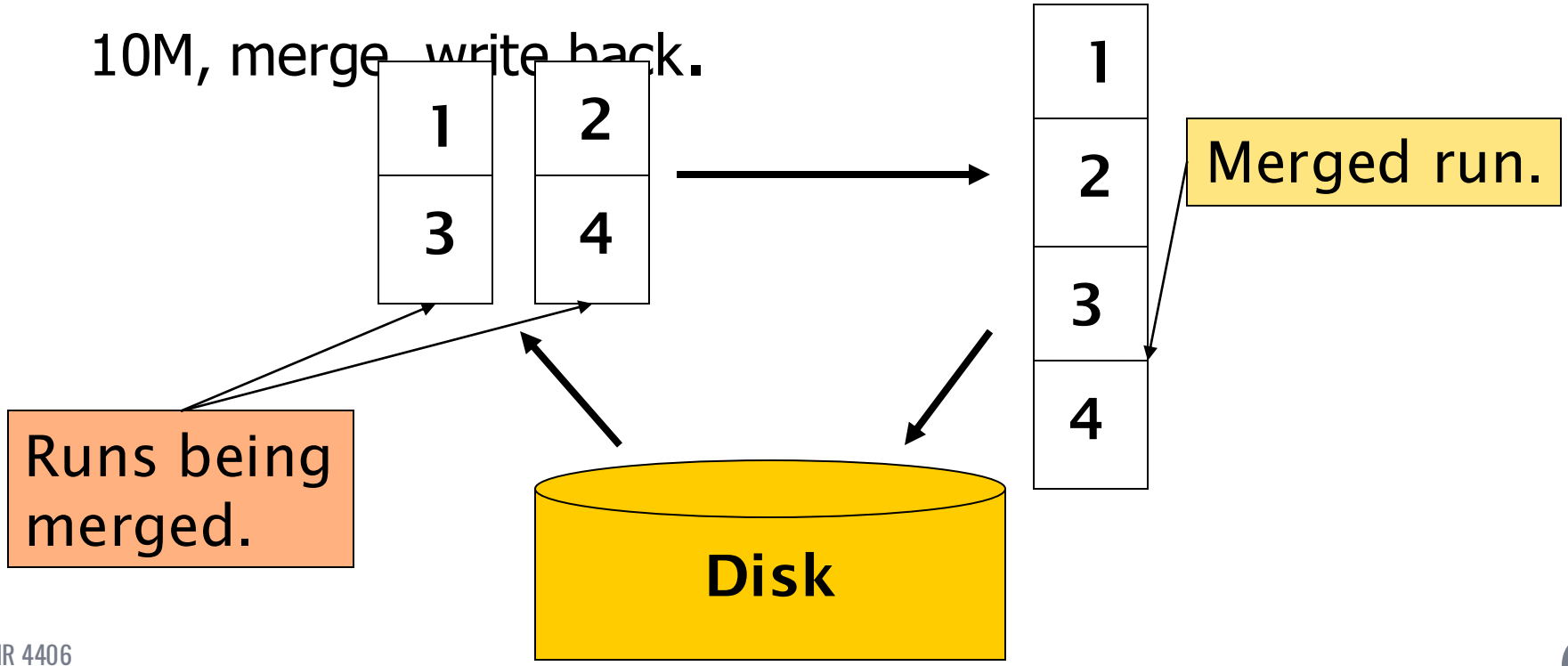
Sorting 10 blocks of 10M records

- First, read each block and sort within:
  - Quicksort takes $2N \ln N$ expected steps
  - In our case 2 x (10M ln 10M) steps

```
BSBIndexConstruction()
n ← 0
while (all documents have not been processed)
do n ← n + 1
    block ← ParseNextBlock()
    BSBI-Invert(block)
    WriteBlockToDisk(block, f_n)
MergeBlocks(f_1, ..., f_n; f_merged)
```

How to merge the sorted runs?

- Can do binary merges, with a merge tree of $\log_2 10 = 4$ layers.
- During each layer, read into memory runs in blocks of 10M, merge, write back.



How to merge the sorted runs?

- But it is more efficient to do a multi-way merge, where you are reading from all blocks simultaneously
- Providing you read decent-sized chunks of each block into memory and then write out a decent-sized output chunk, then you're not killed by disk seeks

© Nir Grinberg

Remaining problem with sort-based algorithm

- Our assumption was: we can keep the dictionary in memory.
- We need the dictionary (which grows dynamically) in order to implement a term to termID mapping.
- Actually, we could work with term, docID postings instead of termID, docID postings .
- . . . but then intermediate files become very large. (We would end up with a scalable, but very slow index construction method.)

SPIMI: Single-pass in-memory indexing

- Key idea 1: Generate separate dictionaries for each block – no need to maintain term-termID mapping across blocks.
- Key idea 2: Don't sort. Accumulate postings in postings lists as they occur.
- With these two ideas we can generate a complete inverted index for each block.
- These separate indexes can then be merged into one big index.

© Nir Grinberg

SPIMI-Invert

```
SPIMI-INVERT(token_stream)
  output_file = NEWFILE()
  dictionary = NEWHASH()
  while (free memory available)
  do token ← next(token_stream)
     if term(token) ∉ dictionary
       then postings_list = ADDTODICTIONARY(dictionary, term(token))
       else postings_list = GETPOSTINGSLIST(dictionary, term(token))
     if full(postings_list)
       then postings_list = DOUBLEPOSTINGSLIST(dictionary, term(token))
     ADDTOPOSTINGSLIST(postings_list, docID(token))
  sorted_terms ← SORTTERMS(dictionary)
  WRITEBLOCKTODISK(sorted_terms, dictionary, output_file)
  return output_file
```

- Merging of blocks is analogous to BSBI.

ANY QUESTIONS ABOUT BSBI OR SPIMI?

© Nir Grinberg

Outline

~~Text operations~~

- ~~Normalization~~
  - ~~Stemming methods~~

Indexing

1. ~~Simple index construction~~
2. ~~Massive data indexing~~
3. Distributed & dynamic indexing

Distributed indexing

- For web-scale indexing must use a distributed computing cluster
- Individual machines are fault-prone
  - Can unpredictably slow down or fail
- How do we exploit such a pool of machines?

© Nir Grinberg

Web search engine data centers

- Web search data centers (Google, Bing, Baidu) mainly contain commodity machines.
- Data centers are distributed around the world.
- Estimate: Google ~1 million servers, 3 million processors/cores (Gartner 2007)
- July 2016 Gartner estimates that Google at the time had 2.5 million servers.
- https://www.datacenterknowledge.com/archives/2017/03/16/google-data-center-faq

Massive data centers

- If in a non-fault-tolerant system with 1000 nodes, each node has 99.9% uptime, what is the uptime of the system?
- Answer: 36.7%
- https://www.youtube.com/watch?v=XZmGGAbHqa0&t=44s

© Nir Grinberg

Distributed indexing

- Maintain a *master* machine directing the indexing job – considered "safe".
- Break up indexing into sets of (parallel) tasks.
- Master machine assigns each task to an idle machine from a pool.

© Nir Grinberg

Parallel tasks

- We will use two sets of parallel tasks
  - Parsers
  - Inverters
- Break the input document collection into *splits*
- Each split is a subset of documents (corresponding to blocks in BSBI/SPIMI)

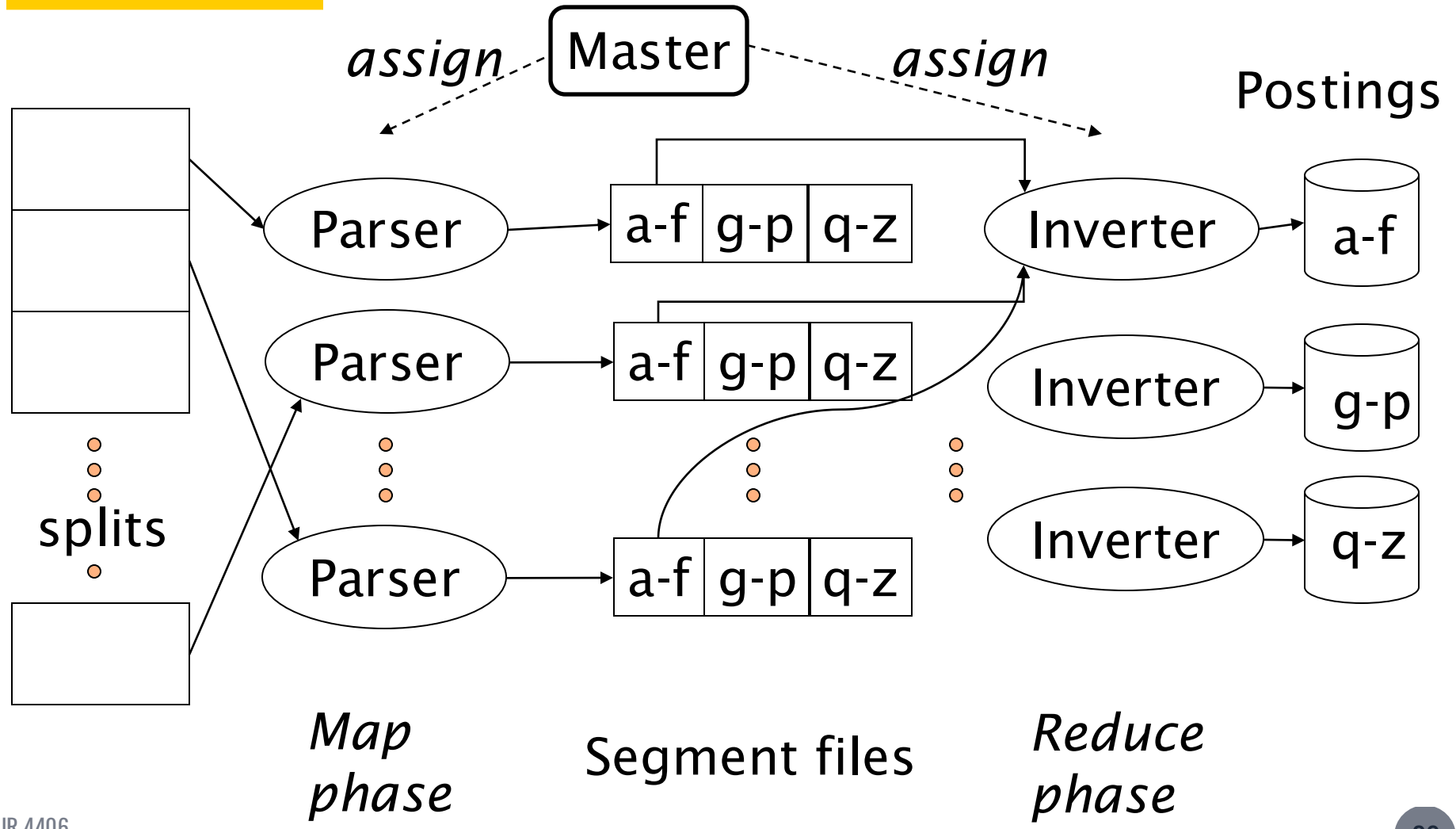
Parsers

- Master assigns a split to an idle parser machine
- Parser reads a document at a time and emits (term, doc) pairs
- Parser writes pairs into *j* partitions
- Each partition is for a range of terms' first letters
  - (e.g., ***a-f, g-p, q-z***) – here *j* = 3.
- Now to complete the index inversion

Inverters

- An inverter collects all (term,doc) pairs (= postings) for <u>one</u> term-partition.
- Sorts and writes to postings lists

Data flow


assign
Master
assign
Postings
Parser
a-f
g-p
q-z
Inverter
a-f
Parser
a-f
g-p
q-z
Inverter
g-p
splits
Parser
a-f
g-p
q-z
Inverter
q-z
Map phase
Segment files
Reduce phase

MapReduce

- The index construction algorithm we just described is an instance of *MapReduce*.
- MapReduce (Dean and Ghemawat 2004) is a robust and conceptually simple framework for distributed computing …
- … without having to write code for the distribution part.
- They describe the Google indexing system (ca. 2002) as consisting of a number of phases, each implemented in MapReduce.

© Nir Grinberg

Schema for index construction in MapReduce

- **Schema of map and reduce functions**
- map: input → list(k, v) reduce: (k,list(v)) → output
- **Instantiation of the schema for index construction**
- map: collection → list(termID, docID)
- reduce: (<termID1, list(docID)>, <termID2, list(docID)>, …) → (postings list1, postings list2, …)

Example for index construction

- Map:
  - Input:
    - d1 : C came, C went.
    - d2 : C died.
  - Output: <C,d1>, <came,d1>, <C,d1>, <went, d1>, <C, d2>, <died,d2>
- Reduce:
  - Input: (<C,(d1,d2,d1)>, <died,(d2)>, <came,(d1)>, <went,(d1)>)
  - Output: (<C,(d1:2,d2:1)>, <died,(d2:1)>, <came,(d1:1)>, <went,(d1:1)>)

© Nir Grinberg

Dynamic indexing

- Up to now, we have assumed that collections are static.
- They rarely are:
  - Documents come in over time and need to be inserted.
  - Documents are deleted and modified.
- This means that the dictionary and postings lists have to be modified:
  - Postings updates for terms already in dictionary
  - New terms added to dictionary

Simplest approach

- Maintain "big" main index
- New docs go into "small" auxiliary index
- Search across both, merge results
- Deletions
  - Invalidation bit-vector for deleted docs
  - Filter docs output on a search result by this invalidation bit-vector
- Periodically, re-index into one main index

Issues with main and auxiliary indexes

- Problem of frequent merges – you touch stuff a lot
- Poor performance during merge
- Actually:
  - Merging of the auxiliary index into the main index is efficient if we keep a separate file for each postings list.
  - Merge is the same as a simple append.
  - But then we would need a lot of files – inefficient for OS.
- In reality: Use a scheme somewhere in between (e.g., split very large postings lists, collect postings lists of length 1 in one file etc.)

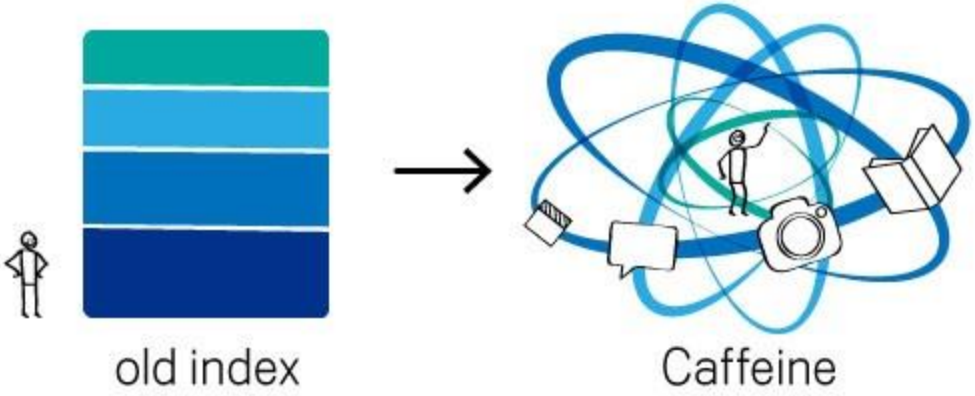
© Nir Grinberg

Further issues with multiple indexes

- Collection-wide statistics are hard to maintain
- E.g., when we spoke of spell-correction: which of several corrected alternatives do we present to the user?
  - We said, pick the one with the most hits
- How do we maintain the top ones with multiple indexes and invalidation bit vectors?
  - One possibility: ignore everything but the main index for such ordering
- Will see more such statistics used in results ranking

© Nir Grinberg

Google dynamic indexing



Dynamic indexing at search engines

- All the large search engines now do dynamic indexing
- Their indices have frequent incremental changes
  - News items, blogs, new topical web pages
    - Sarah Palin, COVID-19, …
- But (sometimes/typically) they also periodically reconstruct the index from scratch
  - Query processing is then switched to the new index, and the old index is deleted

© Nir Grinberg

Other sorts of indexes

- Positional indexes
  - Same sort of sorting problem … just larger
- Building character n-gram indexes:
  - As text is parsed, enumerate *n*-grams.
  - For each *n*-gram, need pointers to all dictionary terms containing it – the "postings".
  - Note that the same "postings entry" will arise repeatedly in parsing the docs – need efficient hashing to keep track of this.
    - E.g., that the trigram <u>*uou*</u> occurs in the term ***deciduous*** will be discovered on each text occurrence of ***deciduous***
    - Only need to process each term once

© Nir Grinberg

Outline

Text operations

- Normalization
  - Stemming methods

Indexing

1. Simple index construction
2. Massive data indexing
3. Distributed & dynamic indexing